

B2B SaaS Platform Testing - Multi-Platform Automation

Overview

You're joining the QA team at "WorkFlow Pro" - a B2B project management SaaS platform. The platform serves multiple clients (multi-tenant), works across web browsers and mobile devices, and integrates with various third-party services.

Time Allocation

- **Take-Home Portion:** 90 minutes maximum
 - **Live Discussion:** 30-45 minutes (scheduled separately)
-

Part 1: Debugging Flaky Test Code (30 minutes)

A previous intern wrote this Playwright test for user login functionality. **The test is flaky** - sometimes passes, sometimes fails in CI/CD pipeline.

```
import pytest
from playwright.sync_api import sync_playwright

def test_user_login():
    with sync_playwright() as p:
        browser = p.chromium.launch()
        page = browser.new_page()

        # Navigate to login page
        page.goto("https://app.workflowpro.com/login")

        # Fill login form
        page.fill("#email", "admin@company1.com")
        page.fill("#password", "password123")
        page.click("#login-btn")

        # Verify successful login
        assert page.url == "https://app.workflowpro.com/dashboard"
        assert page.locator(".welcome-message").is_visible()

    browser.close()

def test_multi_tenant_access():
    with sync_playwright() as p:
        browser = p.chromium.launch()
        page = browser.new_page()
```

```
page.goto("https://app.workflowpro.com/login")
page.fill("#email", "user@company2.com")
page.fill("#password", "password123")
page.click("#login-btn")

# User should only see Company2 data
projects = page.locator(".project-card").all()
for project in projects:
    assert "Company2" in project.text_content()

browser.close()
```

Your Tasks:

1. **Identify Flakiness Issues:** List all problems that could cause intermittent failure

The test clicks the login button and immediately verifies the URL without waiting for the page to finish loading. Since the dashboard loads dynamically, checking the welcome message immediately can also lead to intermittent failures. The login flow does not handle the optional 2FA step mentioned in the requirements, which may stop the test before reaching the dashboard. In the multi-tenant test, project cards are fetched immediately after login without ensuring that the data has finished loading. The test also relies on hardcoded credentials and URLs, making it less flexible across different environments.

2. **Explain Root Causes:** Why do these issues happen in CI/CD vs local environment?

These issues are more common in CI/CD because applications generally take longer to load due to shared infrastructure, network latency, and cross-browser execution. On a local machine, pages may load fast enough that these synchronization problems are not visible. Since the application uses dynamic loading and different tenants can have different response times, immediate assertions make the tests unreliable and lead to inconsistent results.

3. **Provide Fixes:** Write the corrected version with proper waits, error handling, and reliability improvements

To improve reliability, I would use Playwright's built-in waiting mechanisms such as `wait_for_url()` and `expect()` instead of performing immediate assertions. I would wait for important UI elements before validating them, handle the optional 2FA flow when required, and replace hardcoded values with configurable test data. Browser cleanup should also be handled using `try...finally` or pytest fixtures so that resources are released even if the test fails.

Additional Context (you may need to ask for more):

- App uses dynamic loading for dashboard elements
 - Login process includes 2FA for some users
 - Different tenants have different loading times
 - CI runs on different browsers and screen sizes
-

Part 2: Test Framework Design (25 minutes)

Design a test automation framework structure for this B2B SaaS platform. **Note:** Requirements are intentionally incomplete.

Given Requirements:

- Support web (Chrome, Firefox, Safari) and mobile (iOS, Android) testing
- Handle multiple tenant environments (company1.workflowpro.com, company2.workflowpro.com)
- Test different user roles (Admin, Manager, Employee) with varying permissions
- API testing for backend services
- Integration with BrowserStack for cross-platform testing
- CI/CD pipeline integration

Your Tasks:

1. **Framework Structure:** Design folder structure, base classes, and utilities

To make the framework easy to maintain and scale, I would separate the project into different modules based on their responsibilities. UI test cases would be placed under the `tests` folder, while the `pages` folder would contain Page Object Model (POM) classes to keep UI interactions reusable. API-related methods would be maintained separately in the `api` folder, and common helper functions such as logging, utilities, and reusable methods would be placed inside the `utils` folder. Test data and environment-specific configurations would also be maintained separately to avoid hardcoding values in the test scripts.

AutomationFramework/ - root directory

tests/

ui/

api/

mobile/

integration/

pages/

api/

utils/

config/

test_data/

reports/

[conftest.py](#)

pytest.ini

requirements.txt

2. **Configuration Management:** How to handle multiple environments, browsers, and test data

I would manage different environments such as Development, QA, and Production using configuration files or environment variables instead of hardcoding URLs and credentials. Browser selection Chrome, tenant information, user roles, and BrowserStack credentials should also be configurable so the same test suite can run across different environments without modifying the test code. Test data should be maintained separately to make the tests reusable and easier to maintain.

3. **Identify Missing Requirements:** What questions would you ask about test data management, reporting, parallel execution, etc.?

How will test data be created and cleaned up after execution?

Is there a dedicated environment for automation testing

How should 2FA be handled for automated test users?

Should the framework support parallel execution, and if yes, what is the expected level of concurrency?

Format: Use text/pseudocode/diagrams to explain your design

Part 3: API + UI Integration Test (35 minutes)

Create a test that validates the complete flow of project creation through both API and UI layers.

Business Scenario:

1. Create a project via API
2. Verify project appears correctly in web UI
3. Check project is accessible on mobile
4. Validate proper tenant isolation (project only visible to correct company)

API Endpoint Information:

POST /api/v1/projects

Headers: Authorization: Bearer {token}, X-Tenant-ID: {company_id}

Body: {"name": "Test Project", "description": "...", "team_members": [...]}

Response: {"id": 123, "name": "Test Project", "status": "active"}

Your Tasks:

1. **Write Integration Test:** Combine pytest (API) + Playwright (UI) + BrowserStack mobile testing
2. **Handle Test Data:** Show how you'd manage test data across API/UI
3. **Cross-Platform Validation:** Ensure the test works on multiple browsers and mobile devices
4. **Tenant Isolation:** Verify security boundaries between companies

Expected Test Structure:

```
def test_project_creation_flow():
```

```
# 1. API: Create project
# 2. Web UI: Verify project display
# 3. Mobile: Check mobile accessibility
# 4. Security: Verify tenant isolation
pass
```

Your Tasks:

1. **Implement the test** using pytest + Playwright + BrowserStack concepts

My Testing approach would first validate the complete project creation flow across different layers of the application. Instead of creating the project through the UI, I would use the API to create the test data because it is faster, more reliable, and reduces unnecessary UI interactions. After successful creation, I would verify that the project is displayed correctly in the web application using Playwright. The same validation would then be executed on Android and iOS devices through BrowserStack to ensure consistent behavior across platforms. Finally, I would verify tenant isolation by confirming that the project is only accessible to the tenant that created it. As part of test cleanup, the created project would be deleted using the API to avoid leaving unnecessary test data in the environment.

```
import uuid

import requests

from playwright.sync_api import expect

BASE_URL = "https://app.workflowpro.com"

def test_project_creation_flow(page):
    project_name = f"Automation-{uuid.uuid4()}"

    token = get_auth_token()

    headers = {
        "Authorization": f"Bearer {token}",
        "X-Tenant-ID": "company1"
    }

    payload = {
        "name": project_name,
        "description": "Automation Test Project",
        "team_members": [101, 102]
    }
```

```

response = requests.post(
    f'{BASE_URL}/api/v1/projects',
    headers=headers,
    json=payload
)
assert response.status_code == 201
project_id = response.json()["id"]
page.goto("https://company1.workflowpro.com/dashboard")
page.wait_for_load_state("networkidle")

expect(
    page.locator(f"text={project_name}")
).to_be_visible()
logout()
login_as_company2()
expect(
    page.locator(f"text={project_name}")
).not_to_be_visible()
requests.delete(
    f'{BASE_URL}/api/v1/projects/{project_id}',
    headers=headers
)

```

2. **Handle edge cases** (network failures, slow loading, mobile responsiveness)

To improve the reliability of the test, I would handle a few common edge cases. API failures such as temporary network issues would be handled with retry logic before marking the test as failed. Since the dashboard loads dynamically, Playwright's built-in waiting methods (`wait_for_load_state()` and `expect()`) would be used instead of fixed delays. Mobile validation would be executed on multiple devices through BrowserStack to identify responsiveness or device-specific issues. The test data would be generated dynamically to avoid conflicts with existing projects, and cleanup would always be performed after execution to keep the test environment reusable.

Assumptions:

Authentication tokens are generated before the test execution.

Dedicated automation accounts exist for each tenant.

BrowserStack is already configured for Android and iOS devices.

The API supports deleting test data after execution.

Optional 2FA is disabled or bypassed for automation users.

3. **Explain approach** with comments on your testing strategy

I chose an API-first approach because creating test data through the API is faster and more stable than creating it through the UI. The UI is then used only to validate business functionality, while BrowserStack ensures that the same functionality works consistently across supported mobile devices. Finally, tenant isolation is verified to confirm that users can access only the data belonging to their own organization, which is a key security requirement for a multi-tenant SaaS application.

Hints: You'll need to make assumptions about authentication, test data cleanup, and mobile testing setup.

Submission Instructions

1. **Create a document** with your responses to all three parts
 2. **Include reasoning** for each technical decision
 3. **List assumptions** about missing requirements or unclear specifications
 4. **Show test framework thinking** - not just individual test cases
 5. **Be prepared** to discuss testing strategy and tool choices in live session
-

Live Session Topics (Preview)

During our video call, we'll discuss:

- Your approach to debugging flaky tests and prevention strategies
 - Framework design decisions and scalability considerations
 - How you'd handle test data management in multi-tenant environments
 - BrowserStack integration strategy and cost optimization
 - CI/CD pipeline integration and parallel execution
 - Monitoring and reporting for test automation
-

Evaluation Criteria

Technical Skills:

- Understanding of test automation tools (Playwright, pytest, BrowserStack)
- Knowledge of web/mobile testing challenges
- API testing and integration concepts
- Framework design and maintainability thinking

Testing Strategy:

- Ability to identify and fix flaky tests
- Understanding of multi-tenant testing challenges
- Cross-platform testing approach
- Test data management strategies

Problem-Solving:

- Systematic debugging approach
- Recognition of missing requirements
- Consideration of real-world constraints (CI/CD, cost, maintenance)

Communication:

- Clear explanation of testing decisions
- Ability to ask clarifying questions about ambiguous requirements
- Understanding of business impact of testing choices

Note: This assessment evaluates your current automation skills and learning potential. We value thoughtful approaches over perfect implementations - focus on demonstrating your testing mindset and technical reasoning.